Universidade de São Paulo



Web-App do SIBiSC

SSC0961 - Desenvolvimento Web e Mobile

Eduardo Paz e Silva	Nº USP: 15506648
Matheus Marchi Baron	Nº USP: 14598431
Pedro Augusto Pereira Magalhães	Nº USP: 11802544
Pedro Dorigatti Aureo Ferreira	Nº USP: 15483592
Rafael Feltrim	Nº USP: 15942812

22/06/2026

Sumário

1 Introdução **2**
1.1 Problema Existente . . . 2
1.2 Solução . . . 2
1.3 Público Alvo . . . 2

2 Funcionalidades e Interface **3**
2.1 Notícias . . . 3
2.2 Eventos . . . 3
2.3 Acervo e Geolocalização . . . 4
2.4 Engajamento . . . 4
2.5 Operação Interna . . . 4
2.6 Interfaces . . . 4
2.6.1 Tela Inicial . . . 4
2.6.2 Tela de Notícias . . . 5
2.6.3 Tela de Eventos . . . 5
2.6.4 Tela de Catálogo . . . 5
2.6.5 Tela de Perfil . . . 6

3 Framework **6**
3.1 Análise de 3 frameworks . . . 6
3.2 Framework implementado . . . 6

4 Desenvolvimento do App **7**
4.1 Tecnologias Utilizadas . . . 7
4.1.1 Front-end . . . 7
4.1.2 Back-end . . . 7
4.1.3 Qualidade e Integração (Outras Tecnologias) . . . 7
4.2 Estrutura do Código . . . 7
4.2.1 `src/components/` . . . 8
4.2.2 `src/pages/` . . . 8
4.2.3 `src/services/` . . . 8
4.2.4 `src/routes/` . . . 8
4.2.5 `src/contexts/` e `src/hooks/` . . . 8
4.2.6 Arquivos e Diretórios de Infraestrutura . . . 8
4.3 Integração com Banco de Dados . . . 9
4.3.1 Explicação do código e da arquitetura de integração . . . 9
4.3.2 Estruturação, segurança e regras de negócio no banco . . . 9
4.4 Repositório Github . . . 9
4.5 Vídeo Demo . . . 10

5 Testes de Validação e Mitigação de Vulnerabilidades **10**
5.1 Gerenciamento Seguro de Sessão e Mitigação de XSS . . . 10
5.2 Controle de Acesso e Row Level Security (RLS) . . . 10
5.3 Sistema de Perfis de Acesso (*Roles*) . . . 10
5.4 Proteção de Rotas e Validação de Entradas . . . 10
5.4.1 Rotas Protegidas . . . 10
5.4.2 Validações no Cliente . . . 10
5.4.3 Casos de Teste — Validações no Cliente . . . 12
5.4.4 Contenção de Tentativas Excessivas . . . 14
5.5 Cabeçalhos de Segurança . . . 14

6 Uso de IA Generativa **14**

7 Discussões **14**
7.1 Dificuldades e Desafios . . . 14
7.2 Aprendizados . . . 15

A Link da Apresentação em PowerPoint **16**

1 Introdução

1.1 Problema Existente

O Sistema Integrado de Bibliotecas de São Carlos - *SIBiSC* - é o responsável pelo gerenciamento e pelo manejamento das bibliotecas públicas do município de São Carlos. Dentre suas funções mais específicas, destacam-se:

a) realização de eventos;

b) organização do acervo municipal;

c) democratização da informação;

d) garantia da integridade dos bancos de dados entre as bibliotecas.

Atualmente, já existem projetos que tentem integrar o *SIBiSC* à Internet e ao mundo digital, como o website *educasaocarlos* e o sistema de busca online *sibi*, os quais são responsáveis por expor informações sobre o sistema municipal e sobre o catálogo de livros e obras disponíveis nas distintas unidades. Contudo, observa-se que a descentralização dos serviços, a obsolescência da interface e a limitada presença em plataformas móveis representam obstáculos que demandam soluções alinhadas às tecnologias atuais de desenvolvimento.

1.2 Solução

a) **Portal de Notícias:** Uma interface dinâmica no estilo *Newsletter*. Além de produzir conteúdo autoral sobre a literatura e história de São Carlos, o portal atuará como um agregador de mídias externas, mantendo o cidadão informado sobre o ecossistema cultural da região.

b) **Calendário de Eventos:** Sistema centralizado de eventos do SIBiSC. Permitirá ao usuário visualizar oficinas, clubes de leitura e palestras, com funcionalidades de inscrição direta e lembretes integrados à agenda do dispositivo.

c) **Consulta de Disponibilidade:** Ferramenta de busca em tempo real na base de dados do SIBiSC. O sistema utilizará geolocalização para apontar em qual das unidades municipais o livro desejado está disponível para empréstimo.

1.3 Público Alvo

O público alvo se subdivide nas personas de 1 a 6, dentre as quais se prioriza atender às demandas das personas de 1 a 3 (consumidores do produto) antes de se atender às demandas de 4 a 6 (mantenedores e gestores do produto).

a) **Persona 1 - Cidadão interessado em cultura local:**

- Perfil: morador de São Carlos que quer acompanhar atividades, oficinas, notícias e funcionamento das bibliotecas;
- Necessidade central: descobrir rapidamente o que está acontecendo no ecossistema cultural local;
- Dor atual: informações espalhadas entre redes sociais, comunicados e portais institucionais difíceis de navegar;
- Valor esperado do produto: acesso simples a notícias e agenda em uma interface clara e atualizada.

b) **Persona 2 - Estudante ou pesquisador:**

- Perfil: usuário que busca livros e precisa localizar onde há disponibilidade de forma objetiva;
- Necessidade central: encontrar um título, entender em qual unidade ele está e decidir se vale o deslocamento;
- Dor atual: ausência de experiência mobile eficiente e dificuldade em consolidar dados de acervo;
- Valor esperado do produto: busca rápida, disponibilidade por unidade e indicação da unidade mais próxima.

c) **Persona 3 - Participante de eventos:**

- Perfil: usuário que deseja participar de oficinas, encontros, clubes de leitura ou ações culturais;
- Necessidade central: descobrir eventos, entender regras e não perder a data;

- Dor atual: baixa previsibilidade da agenda e pouca lembrança ativa;
- Valor esperado do produto: visão de calendário, detalhe do evento e lembretes compatíveis com ambiente web.

d) **Persona 4 - Bibliotecário:**

- Perfil: profissional responsável por operação local, atualização de dados e confiabilidade do acervo;
- Necessidade central: manter informações de unidade, horário e disponibilidade corretas;
- Dor atual: dados fragmentados, retrabalho e atualização descentralizada;
- Valor esperado do produto: base unificada e fluxo interno claro para operação mínima.

e) **Persona 5 - Curador de conteúdo:**

- Perfil: pessoa responsável por notícias, agenda e revisão de informações públicas;
- Necessidade central: organizar, publicar e corrigir conteúdos com segurança;
- Dor atual: falta de canal centralizado e baixa padronização editorial;
- Valor esperado do produto: curadoria simples e previsível para notícias e eventos.

f) **Persona 6 - Gestão SIBiSC:**

- Perfil: coordenação que precisa garantir continuidade, aderência institucional e valor público;
- Necessidade central: acompanhar riscos, uso do produto e saúde operacional;
- Dor atual: dificuldade em medir engajamento e impacto da comunicação cultural;
- Valor esperado do produto: visão consolidada de operação, dados e evolução do projeto.

2 Funcionalidades e Interface

As funcionalidades do *web-app* do SIBiSC são divididas conforme as necessidades principais do público alvo da aplicação, no formato de *user stories*:

2.1 Notícias

a) US-NOT-001 - Como cidadão, quero visualizar as notícias mais recentes do SIBiSC para me manter informado sobre o ecossistema cultural local. Persona: Cidadão

b) US-NOT-002 - Como cidadão, quero abrir o detalhe de uma notícia para ler o conteúdo completo com contexto e origem. Persona: Cidadão

c) US-NOT-003 - Como cidadão, quero filtrar notícias por categoria ou tema para localizar assuntos relevantes com menos esforço. Persona: Cidadão

d) US-NOT-004 - Como curador, quero revisar, editar e publicar notícias para garantir consistência editorial e confiabilidade das informações públicas. Persona: Curador de conteúdo

2.2 Eventos

a) US-EVT-001 - Como participante, quero ver a agenda de eventos em formato de lista e calendário para me organizar com antecedência. Persona: Participante de eventos

b) US-EVT-002 - Como participante, quero abrir o detalhe de um evento para entender local, horário, descrição e público-alvo. Persona: Participante de eventos

c) US-EVT-003 - Como participante, quero adicionar um evento à minha agenda para não esquecer o compromisso. Persona: Participante de eventos

d) US-EVT-004 - Como curador, quero cadastrar, editar e cancelar eventos para manter a agenda pública confiável. Persona: Curador de conteúdo

2.3 Acervo e Geolocalização

a) US-ACV-001 - Como estudante, quero buscar livros por título, autor ou palavra-chave para localizar rapidamente obras de interesse. Persona: Estudante/Pesquisador

b) US-ACV-002 - Como estudante, quero ver a disponibilidade de um livro por unidade para decidir onde realizar a busca física. Persona: Estudante/Pesquisador

c) US-ACV-003 - Como estudante, quero identificar a unidade mais próxima com o livro disponível para reduzir meu esforço de deslocamento. Persona: Estudante/Pesquisador

d) US-ACV-004 - Como bibliotecário, quero atualizar dados operacionais do acervo para manter a consulta pública coerente com a realidade. Persona: Bibliotecário

2.4 Engajamento

a) US-ENG-001 - Como cidadão, quero favoritar livros e eventos para voltar facilmente ao que me interessa. Persona: Cidadão

b) US-ENG-002 - Como cidadão, quero receber lembretes e notificações sobre conteúdos ou eventos relevantes para manter engajamento ativo. Persona: Cidadão

c) US-ENG-003 - Como gestão, quero acompanhar sinais de interesse do público para orientar decisões futuras de comunicação e operação. Persona: Gestão SIBiSC

2.5 Operação Interna

a) US-OPS-001 - Como bibliotecário, quero gerenciar unidades, horários e informações públicas para manter o serviço atualizado. Persona: Bibliotecário

b) US-OPS-002 Como curador, quero validar conteúdos importados de fontes externas para evitar publicação de dados incorretos. Persona: Curador de conteúdo

c) US-OPS-003 - Como gestão, quero consultar o histórico de sincronizações e falhas para acompanhar a saúde operacional do sistema. Persona: Gestão SIBiSC

2.6 Interfaces

A interface do aplicativo foi prototipada de forma interativa utilizando a ferramenta *Figma*, buscando atender às necessidades identificadas durante a definição das personas e dos requisitos do sistema. O protótipo prioriza uma experiência adaptada para dispositivos móveis, com navegação simplificada e acesso rápido às funcionalidades mais utilizadas pelos usuários. O protótipo completo pode ser acessado por meio do seguinte link: Interface do Aplicativo. Ressalta-se que a versão disponível no Figma não inclui a tela de perfil do usuário, uma vez que essa funcionalidade foi definida apenas na segunda etapa do projeto, após refinamentos dos requisitos e discussões realizadas pela equipe de desenvolvimento.

A estrutura principal da aplicação é composta por cinco telas, descritas a seguir.

2.6.1 Tela Inicial

A tela inicial foi projetada para servir como ponto central de acesso às principais funcionalidades do sistema. Nela são apresentados resumos das notícias mais recentes publicadas pelo SIBiSC e destaques dos próximos eventos cadastrados nas bibliotecas municipais. Dessa forma, o usuário pode acompanhar rapidamente as novidades do ecossistema cultural de São Carlos sem a necessidade de navegar por diferentes seções da aplicação.
Essa interface atende principalmente à Persona 1 (Cidadão interessado em cultura local), que busca uma maneira simples e centralizada de se manter informado sobre atividades, eventos e notícias relacionadas às bibliotecas públicas do município.



Figura 1: Protótipo da tela inicial.

2.6.2 Tela de Notícias

A tela de notícias concentra os conteúdos editoriais disponibilizados pelo sistema. Seu objetivo é oferecer um canal centralizado de divulgação de informações relacionadas ao SIBiSC, às bibliotecas municipais e ao cenário cultural da cidade. Além de conteúdos produzidos pela própria equipe responsável pela plataforma, a interface também foi concebida para acomodar notícias provenientes de fontes externas previamente selecionadas e validadas.
A funcionalidade atende principalmente à Persona 1, permitindo que o cidadão acompanhe informações relevantes de forma organizada e acessível em dispositivos móveis.



Figura 2: Protótipo da tela de notícias.

2.6.3 Tela de Eventos

A tela de eventos centraliza a agenda cultural das bibliotecas municipais. Por meio dela, os usuários podem consultar oficinas, palestras, clubes de leitura e demais atividades promovidas pelas unidades participantes do SIBiSC. Cada evento apresenta informações detalhadas, como data, horário, local e descrição.
A proposta busca atender principalmente à Persona 3 (Participante de eventos), oferecendo maior previsibilidade da agenda e facilitando o acompanhamento das atividades disponíveis.



Figura 3: Protótipo da tela de eventos.

2.6.4 Tela de Catálogo

A tela de catálogo disponibiliza mecanismos de busca para consulta ao acervo das bibliotecas municipais. O usuário pode pesquisar livros utilizando critérios como título, autor ou palavras-chave e acessar informações sobre disponibilidade e localização da obra nas diferentes unidades do sistema.
Essa funcionalidade atende principalmente à Persona 2 (Estudante ou pesquisador), cuja principal necessidade consiste em localizar rapidamente obras de interesse e identificar a unidade mais adequada para realizar o empréstimo.



Figura 4: Protótipo da tela de catálogo.

2.6.5 Tela de Perfil

A tela de perfil concentra as funcionalidades relacionadas à conta do usuário. Nela é possível realizar autenticação, acessar informações pessoais e acompanhar conteúdos de interesse, como livros favoritados, eventos salvos e demais recursos de personalização.
Essa seção foi projetada para aumentar o engajamento dos usuários com a plataforma e oferecer uma experiência mais adequada às preferências individuais de cada cidadão.



Figura 5: Protótipo da tela de perfil.

3 Framework

3.1 Análise de 3 frameworks

Tabela 1: Comparativo entre frameworks para desenvolvimento mobile

Característica	**React Native**	**Flutter**	**Ionic / Capacitor**
Plataformas	iOS, Android, Web, Windows, macOS	iOS, Android, Web, Windows, macOS, Linux	iOS, Android, Web (PWA), Desktop (Electron)
Linguagens	JavaScript, TypeScript	Dart	HTML, CSS, JavaScript (React, Vue, Angular)
Vantagens	Reuso de código Web (React), performance próxima ao nativo, vasto ecossistema.	UI consistente (pixel-perfect), alta performance, excelente documentação.	Curva de aprendizado baixíssima para devs web, ideal para PWAs e prototipagem rápida.
Desvantagens	Dependência de pontes (*bridges*) nativas; atualizações complexas.	Linguagem Dart possui mercado menor que JS; apps tendem a ser maiores.	Performance inferior em apps complexos (roda em WebView).
Curva	Média (Baixa p/ React)	Média	Baixa

Fonte: Elaborada pelo autor (2026).

3.2 Framework implementado

O *framework* escolhido foi o *react native*A escolha do *React Native*, em conjunto com o ecossistema Expo, fundamenta-se no fato de o projeto SIBiSC já estar sendo desenvolvido utilizando *React 19* na versão *web*. Essa decisão estratégica promove alinhamento tecnológico entre as plataformas, aumentando a eficiência e a consistência do desenvolvimento.

a) **Reaproveitamento de conhecimento:** A curva de aprendizado é reduzida, uma vez que conceitos como gerenciamento de estado, hooks e consumo de APIs permanecem os mesmos da aplicação web, incluindo integrações já existentes.

b) **Eficiência no desenvolvimento:** A utilização do Expo Go permite testes em dispositivos reais de forma imediata, acelerando o ciclo de desenvolvimento e validação.

c) **Manutenibilidade:** A unificação da lógica de negócio entre web e mobile resulta em uma base de código mais coesa, facilitando manutenção, evolução e correção de erros.

d) **Performance e ecossistema:** O React Native oferece melhor desempenho e experiência de usuário quando comparado ao Ionic, por utilizar componentes nativos em vez de WebView. Além disso, possui um amplo ecossistema de bibliotecas que facilita a implementação de funcionalidades como navegação, autenticação, consumo de APIs e integração com recursos nativos, sendo adequado para uma aplicação que se propõe a ser um hub de informações bibliotecárias e eventos da cidade de São Carlos

e) **Desvantagens do Flutter:** O Flutter apresenta vantagens relevantes, como alta performance e interface consistente entre plataformas. No entanto, sua adoção exigiria o aprendizado da linguagem Dart e adaptação a um novo ecossistema, o que aumentaria a complexidade do projeto considerando o conhecimento atual da equipe.

4 Desenvolvimento do App

4.1 Tecnologias Utilizadas

A arquitetura tecnológica do web-app do SIBiSC foi projetada para garantir alta performance, manutenibilidade e uma experiência de usuário otimizada para dispositivos móveis (mobile-first).

4.1.1 Front-end

Para a construção da interface do usuário e interação do sistema, adotou-se o framework React (versão 19), escolhido devido à sua arquitetura baseada em componentes reutilizáveis e gerenciamento eficiente de estados. Este trabalha em conjunto com o Vite (versão 6), que atua como ferramenta de build e servidor de desenvolvimento, garantindo tempos de inicialização rápidos e otimização do fluxo de trabalho da equipe.

- **Roteamento:** Gerenciado pela biblioteca React Router DOM, que é essencial para permitir uma navegação fluida entre rotas protegidas e públicas sem a necessidade de recarregar a página web.
- **Estilização e acessibilidade:** Utilizou-se CSS Modules para o encapsulamento seguro da estilização, gerando escopos locais para os componentes e prevenindo conflitos visuais. O design segue estritamente uma abordagem mobile-first e atende às diretrizes de acessibilidade e inclusão digital da norma WCAG 2.1AA.

4.1.2 Back-end

Toda a infraestrutura de dados, segurança e autenticação é centralizada no ecossistema Supabase, que atua como plataforma de Backend-as-a-Service (BaaS) integrada.

- **Banco de dados:** PostgreSQL, responsável por hospedar todas as informações do sistema, desde os perfis de usuários até os dados da aplicação, sendo manipulado de forma estruturada para garantir a persistência das informações.
- **Segurança e autenticação:** O sistema gerencia as sessões por meio de JSON Web Tokens (JWT) emitidos pelo Supabase, com políticas de Row Level Security (RLS) garantindo que cada usuário acesse e edite exclusivamente seus próprios dados. Os detalhes da estratégia de proteção de sessão são apresentados na Seção 5.
- **Mock Data:** Durante as fases iniciais de desenvolvimento e testes isolados da interface, a aplicação utilizou uma camada de dados simulados (mock data), posteriormente migrada para o Supabase.

4.1.3 Qualidade e Integração (Outras Tecnologias)

Para manter a integridade do código entre as contribuições da equipe e assegurar a qualidade contínua, o projeto utiliza recursos de integração contínua (CI/CD) geridos através do GitHub Actions. Essa infraestrutura implementa um Quality Gate automatizado, que valida todos os Pull Requests com baterias de testes integrados e funcionais do tipo E2E (End-to-End), executados pela tecnologia Playwright.

4.2 Estrutura do Código

O código-fonte da aplicação foi organizado de forma a facilitar a manutenção, a reutilização de componentes e a separação das diferentes responsabilidades do sistema. Para isso, o projeto foi dividido em diretórios que agrupam arquivos com funções semelhantes, tornando o desenvolvimento mais organizado e escalável.

A principal estrutura da aplicação está localizada no diretório `src/`, cujos módulos são descritos a seguir.

4.2.1 `src/components/`

Este diretório reúne os componentes visuais reutilizáveis da aplicação. Seu objetivo é evitar a repetição de código e manter uma aparência consistente entre as diferentes telas.

Ele é dividido em três grupos principais:

- `cards/`: contém componentes utilizados para exibir informações em formato de cartões, como livros e eventos. Exemplos de arquivos: `BookCard.jsx` e `EventCard.jsx`.
- `layout/`: reúne componentes responsáveis pela estrutura geral da aplicação, como a barra de navegação inferior (`BottomNav.jsx`) e o componente principal de organização das páginas (`AppLayout.jsx`).
- `ui/`: contém elementos de interface utilizados em diferentes partes do sistema, como campos de busca, botões e indicadores de carregamento.

4.2.2 `src/pages/`

Neste diretório estão as páginas da aplicação, responsáveis por combinar os componentes visuais e implementar as funcionalidades de cada tela.

Entre elas destacam-se:

- **Telas de autenticação**: responsáveis pelo login e cadastro de usuários. Antes do envio das informações ao servidor, são realizadas validações para verificar se os campos foram preenchidos corretamente, se o e-mail possui um formato válido e se a senha atende aos requisitos definidos.
- **Telas de listagem e detalhes**: exibem listas de informações, como catálogos e eventos, além de páginas específicas para visualizar os detalhes de cada item selecionado.

4.2.3 `src/services/`

O diretório `services` concentra as funções responsáveis pela comunicação com serviços externos. Dessa forma, os componentes visuais não precisam lidar diretamente com chamadas de rede.

As integrações com o Supabase são realizadas nessa camada, incluindo operações relacionadas ao catálogo, eventos e informações do usuário. Antes da conclusão do backend, essa mesma camada foi populada com dados simulados (mock data), permitindo o desenvolvimento e validação da interface de forma independente da disponibilidade do banco de dados, sendo posteriormente substituída pelas chamadas reais ao Supabase.

4.2.4 `src/routes/`

Este diretório gerencia a navegação entre as páginas da aplicação por meio do React Router DOM.

Além de definir as rotas disponíveis, ele também controla o acesso às áreas restritas. Para isso, foi utilizado o componente `ProtectedRoute`, que verifica se o usuário está autenticado antes de permitir o acesso a determinadas páginas.

4.2.5 `src/contexts/` e `src/hooks/`

Esses diretórios armazenam funcionalidades compartilhadas por diferentes partes da aplicação.

O arquivo `AuthContext.jsx`, por exemplo, é responsável por disponibilizar as informações do usuário autenticado para os demais componentes. O token de autenticação é mantido apenas durante a execução da aplicação, sem armazenamento permanente no navegador, contribuindo para uma maior segurança dos dados do usuário.

4.2.6 Arquivos e Diretórios de Infraestrutura

Além dos módulos responsáveis pelas funcionalidades da aplicação, o projeto possui arquivos voltados para automação, implantação e garantia de qualidade, alguns deles são listados a seguir.

- `.github/`: contém os fluxos de integração contínua configurados com GitHub Actions. Sempre que uma nova alteração é enviada para revisão, testes automatizados são executados para verificar o funcionamento da aplicação.
- `vercel.json`: reúne as configurações utilizadas durante a publicação da aplicação na plataforma Vercel, incluindo definições relacionadas à segurança e ao comportamento do sistema em produção.

- `SECURITY_REPORT.md`: documento utilizado para registrar informações sobre acessibilidade, segurança e verificações realizadas durante o desenvolvimento.

Essa organização permitiu manter o código mais limpo, facilitar futuras manutenções e tornar o desenvolvimento mais eficiente à medida que novas funcionalidades foram adicionadas ao sistema.

4.3 Integração com Banco de Dados

A comunicação entre o front-end e o banco de dados é intermediada diretamente pelas funções da SDK oficial do Supabase. A instância primária do cliente é inicializada de forma centralizada e isolada no arquivo `src/lib/supabaseClient.js`.

Para mitigar a exposição de credenciais sensíveis e seguir boas práticas de segurança, as chaves de conexão são injetadas dinamicamente via variáveis de ambiente gerenciadas pelo Vite (`import.meta.env`), utilizando o operador de encadeamento opcional.

O código abaixo reflete a lógica de inicialização do cliente de banco de dados na aplicação:

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env?.VITE_SUPABASE_URL;
const supabaseKey = import.meta.env?.VITE_SUPABASE_PUBLISHABLE_KEY;

export const isSupabaseConfigured = Boolean(supabaseUrl && supabaseKey);

export const supabase = isSupabaseConfigured
  ? createClient(supabaseUrl, supabaseKey, {
      auth: {
        persistSession: false,
      },
    })
  : null;
```

Código 1: Configuração do cliente Supabase (src/lib/supabaseClient.js)

4.3.1 Explicação do código e da arquitetura de integração

Arquitetura Defensiva: A implementação adota uma validação inicial explícita por meio da variável `isSupabaseConfigured`. Caso o ambiente não esteja provisionado com as variáveis necessárias, o módulo exporta `null` em vez de instanciar um cliente inválido. Essa abordagem evita falhas críticas durante a inicialização e renderização da aplicação.

Gerenciamento de Sessão Seguro: O parâmetro `persistSession: false` desativa a persistência nativa de sessão da SDK do Supabase, de modo que o token de autenticação seja mantido apenas em memória durante a execução da aplicação, em vez de gravado no Local Storage do navegador.

4.3.2 Estruturação, segurança e regras de negócio no banco

Para estruturar o banco de dados e inicializar a aplicação, o sistema realiza a migração dos dados simulados (mock data) para a base de dados hospedada na nuvem. Esse processo é automatizado pela execução do script `seed.sql`, responsável pela inserção inicial dos registros nas tabelas essenciais do sistema.

A integridade e o isolamento dos dados são garantidos por meio das políticas de RLS oferecidas pelo PostgreSQL, que validam individualmente cada operação de leitura e escrita.

Além disso, parte das regras de negócio é implementada diretamente no banco de dados utilizando *triggers* SQL. Durante o processo de cadastro de um novo usuário, esses gatilhos criam automaticamente o perfil correspondente e atribuem a permissão padrão de "cidadão", evitando dependências excessivas da camada de interface.

Por fim, a estrutura relacional do banco sustenta funcionalidades que dependem da persistência de interações dos usuários, como o armazenamento de favoritos no catálogo de livros. Dessa forma, as informações permanecem sincronizadas e disponíveis de maneira confiável entre diferentes sessões e dispositivos.

4.4 Repositório Github

Link do repositório: `https://github.com/RaFeltrim/sibisc-hub-cultural`

4.5 Vídeo Demo

Link do vídeo demo: `https://drive.google.com/file/u/2/d/1FL_Am0EXUpWaFl7hweipgCe2i5dpeiZl/view?usp=sharing`

5 Testes de Validação e Mitigação de Vulnerabilidades

Visando garantir a segurança e a integridade dos dados dos usuários do SIBiSC, foi implementado um fluxo robusto de autenticação e autorização baseado no provedor de identidade do Supabase, utilizando *JSON Web Tokens* (JWT). A arquitetura de segurança foi projetada para mitigar vulnerabilidades comuns em aplicações web, como ataques de *Cross-Site Scripting* (XSS), acessos indevidos e exposição de dados sensíveis.

5.1 Gerenciamento Seguro de Sessão e Mitigação de XSS

Uma prática comum em aplicações web consiste no armazenamento de tokens JWT diretamente no Local Storage do navegador. Entretanto, essa abordagem apresenta riscos significativos relacionados a ataques XSS. Para mitigar esse problema, a SDK do Supabase foi configurada com a diretiva `persistSession: false`, impedindo o armazenamento persistente dos tokens.

Dessa forma, o token de acesso permanece apenas na memória volátil da aplicação React, sendo gerenciado por meio de um `AuthContext` customizado associado ao evento `onAuthStateChange`. Com essa estratégia, ao fechar a aba ou recarregar a aplicação, o token é removido da memória, reduzindo significativamente a superfície de ataque para acesso indevido a informações sensíveis, como o histórico de empréstimos do cidadão.

5.2 Controle de Acesso e Row Level Security (RLS)

Para assegurar que cada usuário tenha acesso exclusivamente aos seus próprios dados, foram habilitadas políticas de *Row Level Security* (RLS) nas tabelas do banco de dados Supabase.

A tabela `user_profiles` foi vinculada à tabela de autenticação `auth.users`, sendo configuradas políticas que permitem operações de consulta (`SELECT`) e atualização (`UPDATE`) apenas quando o identificador do usuário autenticado (`auth.uid()`) corresponde ao identificador do registro solicitado. Essa abordagem garante isolamento dos dados entre usuários e impede acessos não autorizados.

5.3 Sistema de Perfis de Acesso (*Roles*)

Considerando a expansão futura do sistema e a necessidade de diferentes níveis de permissão, foi implementado um mecanismo de perfis de acesso por meio da coluna `role` na tabela `user_profiles`.

Essa coluna aceita apenas os valores: cidadão, curador e admin.

Durante o processo de cadastro de novos usuários pela rota `/cadastro`, um gatilho (*Trigger*) SQL cria automaticamente o perfil correspondente e atribui a função padrão `cidadao`. Como a atribuição ocorre diretamente no banco de dados, o usuário não possui meios de manipular seu nível de acesso durante o cadastro.

5.4 Proteção de Rotas e Validação de Entradas

Diversas medidas foram adotadas para reforçar a segurança da aplicação no nível da interface.

5.4.1 Rotas Protegidas

Foi desenvolvido o componente lógico `<ProtectedRoute />`, responsável por controlar o acesso às páginas restritas do sistema. Ao tentar acessar a rota `/perfil`, a aplicação verifica imediatamente a existência de um JWT válido no contexto global. Caso a autenticação não seja confirmada, o acesso é bloqueado e o usuário é redirecionado para a tela de login.

5.4.2 Validações no Cliente

A aplicação realiza validações preventivas antes do envio das requisições ao servidor (Supabase Auth), reduzindo o consumo desnecessário de recursos e melhorando a experiência do usuário. As validações estão implementadas nas telas de **Login** (`src/pages/LoginPage.jsx`) e **Cadastro** (`src/pages/RegisterPage.jsx`).

Os formulários utilizam o atributo `noValidate`, de modo que a validação nativa do navegador é desativada e todas as regras são aplicadas em JavaScript, no momento do envio (`handleSubmit`). Os erros são exibidos em um único alerta no topo do formulário (`role="alert"`), sem mensagens individuais por campo.

Entre as validações implementadas destacam-se:

- Bloqueio de submissões com campos obrigatórios vazios;
- Verificação de formato de e-mail por meio de expressões regulares (*Regular Expressions*);
- Exigência de senha com no mínimo seis caracteres (apenas na tela de cadastro);
- Verificação de igualdade entre senha e confirmação de senha (apenas na tela de cadastro);
- Validação do nome completo com apenas letras e espaços (apenas na tela de cadastro);
- Indicador visual de força da senha durante a digitação (apenas na tela de cadastro, sem bloqueio de envio).

Tela de Login (`/login`) A tela de login possui dois campos: *E-mail* e *Senha*. A ordem de validação no cliente é a seguinte:

1. Verificar se e-mail e senha foram preenchidos. Caso contrário, exibir: "Por favor, preencha todos os campos."
2. Validar o formato do e-mail com a expressão regular `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`, aplicada após conversão para minúsculas. Caso inválido, exibir: *"Por favor, insira um e-mail válido."*
3. Enviar credenciais ao Supabase via `signInWithPassword`.

Na tela de login, a senha é validada apenas quanto à obrigatoriedade (não vazia). Não há exigência de tamanho mínimo nem de complexidade no momento do acesso.

Após a submissão, erros retornados pelo servidor são traduzidos e exibidos no cliente:

- Credenciais inválidas: *"E-mail ou senha incorretos."*
- Limite de tentativas (*rate limit*): *"Muitas tentativas falhas. Tente novamente mais tarde."*
- Supabase não configurado: *"Supabase is not configured"*
- Demais falhas: mensagem original do erro ou *"Falha ao fazer login."*

Tela de Cadastro (`/cadastro`) A tela de cadastro possui quatro campos: *Nome Completo*, *E-mail*, *Senha* e *Confirmar Senha*. A ordem de validação no cliente é a seguinte:

1. Verificar se todos os campos foram preenchidos (após `trim()` em nome e e-mail). Caso contrário, exibir: *"Por favor, preencha todos os campos."*
2. Validar o nome com a expressão regular `/^[\p{L}\s]+$/u`, permitindo apenas letras (incluindo acentuação) e espaços. Caso inválido, exibir: *"O nome deve conter apenas letras."*
3. Validar o formato do e-mail com a mesma expressão regular da tela de login. Caso inválido, exibir: *"Por favor, insira um e-mail válido."*
4. Verificar se a senha possui no mínimo seis caracteres. Caso contrário, exibir: *"A senha deve ter no mínimo 6 caracteres."*
5. Verificar se a senha e a confirmação são idênticas. Caso contrário, exibir: *"As senhas não coincidem."*
6. Enviar dados ao Supabase via `signUp`, incluindo o nome em `options.data.full_name`.

O campo *Nome Completo* é validado apenas quanto à obrigatoriedade; não há verificação de tamanho mínimo, formato ou composição (nome e sobrenome).

Durante a digitação da senha, um indicador visual de força é exibido com base em critérios de complexidade (comprimento, maiúsculas, números e caracteres especiais). Esse indicador classifica a senha como *Fraca*, *Média* ou *Forte*, mas **não impede** o cadastro — apenas a regra de mínimo de seis caracteres é bloqueante.

Erros retornados pelo servidor após a submissão:

- E-mail já cadastrado: *"Este e-mail já está em uso."*
- Supabase não configurado: *"Supabase is not configured"*
- Demais falhas: mensagem original do erro ou *"Falha ao criar conta."*

A tabela 2 faz um comparativo entre as telas.

Tabela 2: Validações no cliente: Login vs. Cadastro

Validação	**Login**	**Cadastro**
Campos obrigatórios preenchidos	E-mail, Senha	Nome, E-mail, Senha, Confirmação
Formato de e-mail (regex)	Sim	Sim
Senha com mínimo de 6 caracteres	Não	Sim
Senha igual à confirmação	—	Sim
Indicador visual de força da senha	Não	Sim (não bloqueante)
Validação de formato do nome	—	Sim (apenas letras e espaços)

5.4.3 Casos de Teste — Validações no Cliente

Os casos de teste a seguir cobrem as validações preventivas das telas de Login e Cadastro. Cada caso deve ser executado com o frontend em execução (`npm run dev`) e o Supabase configurado no arquivo `.env`, exceto quando indicado.

- **Casos de teste — Tela de Login**

Tabela 3: Casos de teste da tela de Login

ID	**Pré-condição**	**Exemplo**	**Resultado esperado**	**Prior.**
CT-L01	Tela de login aberta	E-mail: `(vazio)`; Senha: `(vazio)`	Alerta: *"Por favor, preencha todos os campos."*	Alta
CT-L02	Tela de login aberta	E-mail: `maria@email.com`; Senha: `(vazio)`	Alerta: *"Por favor, preencha todos os campos."*	Alta
CT-L03	Tela de login aberta	E-mail: `(vazio)`; Senha: `senha123`	Alerta: *"Por favor, preencha todos os campos."*	Alta
CT-L04	Tela de login aberta	E-mail: `emailinvalido`; Senha: `senha123`	Alerta: *"Por favor, insira um e-mail válido."*	Alta
CT-L05	Tela de login aberta	E-mail: `sem@dominio`; Senha: `senha123`	Alerta: *"Por favor, insira um e-mail válido."*	Alta
CT-L06	Tela de login aberta	E-mail: `usuario@exemplo.com`; Senha: `senhaErrada`	Alerta: *"E-mail ou senha incorretos."*	Alta
CT-L07	Usuário cadastrado no Supabase	E-mail: `usuario@exemplo.com`; Senha: `senhaCorreta123`	Redirecionamento para `/perfil` (ou rota de origem)	Alta
CT-L08	Tela de login aberta	E-mail: `teste@email.com`; Senha: `a`	Submissão permitida (sem validação de tamanho mínimo)	Média
CT-L09	Arquivo `.env` ausente ou inválido	E-mail: `teste@email.com`; Senha: `senha123`	Erro: *"Supabase is not configured"* ou falha de conexão	Média
CT-L10	Tela de login aberta	Senha: `minhasenha`; clicar no ícone de olho	Senha alterna entre visível (`minhasenha`) e oculta (••••••••)	Baixa

- **Casos de teste — Tela de Cadastro**

Tabela 4: Casos de teste da tela de Cadastro

ID	Pré-condição	Exemplo	Resultado esperado	Prior.
CT-C01	Tela de cadastro aberta	Nome: `(vazio)`; E-mail: `(vazio)`; Senha: `(vazio)`; Confirmação: `(vazio)`	Alerta: *"Por favor, preencha todos os campos."*	Alta
CT-C02	Tela de cadastro aberta	Nome: `João Silva`; E-mail: `joao@email.com`; Senha: `senha123`; Confirmação: `(vazio)`	Alerta: *"Por favor, preencha todos os campos."*	Alta
CT-C03	Tela de cadastro aberta	Nome: `João Silva`; E-mail: `invalido`; Senha: `senha123`; Confirmação: `senha123`	Alerta: *"Por favor, insira um e-mail válido."*	Alta
CT-C04	Tela de cadastro aberta	Nome: `João Silva`; E-mail: `joao@email.com`; Senha: `abc12`; Confirmação: `abc12`	Alerta: *"A senha deve ter no mínimo 6 caracteres."*	Alta
CT-C05	Tela de cadastro aberta	Nome: `João Silva`; E-mail: `joao@email.com`; Senha: `abcdef`; Confirmação: `abcdeg`	Alerta: *"As senhas não coincidem."*	Alta
CT-C06	E-mail já cadastrado no Supabase	Nome: `Maria Souza`; E-mail: `usuario@exemplo.com`; Senha: `senha123`; Confirmação: `senha123`	Alerta: *"Este e-mail já está em uso."*	Alta
CT-C07	E-mail não cadastrado	Nome: `Pedro Lima`; E-mail: `pednovo@email.com`; Senha: `senha123`; Confirmação: `senha123`	Redirecionamento para `/perfil`	Alta
CT-C08	Tela de cadastro aberta	Senha digitada: `123456` (demais campos válidos)	Barra exibe *"Força da senha: Fraca"*, mas cadastro é permitido	Média
CT-C09	Tela de cadastro aberta	Senha digitada: `Abcdef12!` (demais campos válidos)	Barra exibe *"Força da senha: Forte"*	Média
CT-C10	Tela de cadastro aberta	Nome: ; E-mail: `joao@email.com`; Senha: `senha123`; Confirmação: `senha123`	Alerta: *"Por favor, preencha todos os campos."*	Média
CT-C11	Tela de cadastro aberta	Nome: `123`; E-mail: `joao@email.com`; Senha: `senha123`; Confirmação: `senha123`	Alerta: *"O nome deve conter apenas letras."*	Alta
CT-C12	Tela de cadastro aberta	Nome: `@#$`; E-mail: `joao@email.com`; Senha: `senha123`; Confirmação: `senha123`	Alerta: *"O nome deve conter apenas letras."*	Alta
CT-C13	Tela de cadastro aberta	Senha: `Senha123`; Confirmação: `senha123`	Alerta: *"As senhas não coincidem."*	Média
CT-C14	Tela de cadastro aberta	Senha: `minhasenha`; Confirmação: `outrasenha`; alternar ícones de olho	Cada campo alterna visibilidade de forma independente	Baixa

Critérios gerais de aceite

- Nenhuma requisição ao Supabase deve ser disparada quando uma validação de cliente falhar antes do envio;
- A mensagem de erro exibida deve corresponder exatamente às mensagens documentadas nesta seção;
- O botão de submissão deve permanecer desabilitado apenas durante o estado de carregamento (*loading*), e não por invalidez do formulário;

- Após correção do erro e nova submissão válida, o alerta de erro deve ser removido antes da nova tentativa.

5.4.4 Contenção de Tentativas Excessivas

Para reduzir a efetividade de ataques de força bruta (*Brute Force*), a interface monitora respostas de limitação de requisições retornadas pela API (`HTTP 429 -- Too Many Requests`). Quando múltiplas tentativas inválidas são detectadas, o sistema bloqueia temporariamente novas ações e informa ao usuário a necessidade de aguardar antes de tentar novamente.

5.5 Cabeçalhos de Segurança

O ambiente de produção hospedado na plataforma Vercel foi configurado com cabeçalhos HTTP de segurança definidos no arquivo `vercel.json`. Esses mecanismos contribuem para a mitigação de diferentes categorias de ataques web:

- **X-Frame-Options: DENY**: impede que a aplicação seja incorporada em *iframes*, reduzindo o risco de ataques de *Clickjacking*;
- **X-Content-Type-Options: nosniff**: evita que navegadores realizem inferência de tipos MIME, reduzindo a possibilidade de execução indevida de conteúdos maliciosos;
- **Content-Security-Policy (CSP)**: estabelece políticas restritivas para carregamento de recursos externos, permitindo apenas fontes confiáveis e autorizando a comunicação de dados sensíveis prioritariamente com os domínios da API do Supabase.

Essas medidas, combinadas às políticas de autenticação, autorização e validação implementadas, contribuem significativamente para a redução da superfície de ataque da aplicação e para a proteção dos dados dos usuários do SIBiSC.

6 Uso de IA Generativa

7 Discussões

7.1 Dificuldades e Desafios

Ao longo do desenvolvimento do Web-App do SIBiSC, a equipe enfrentou desafios em diferentes frentes do projeto, desde a definição de requisitos até a implementação técnica da solução.

Um dos primeiros obstáculos esteve relacionado à integração com sistemas legados. Como mencionado na Introdução, o SIBiSC já possui iniciativas digitais parciais, como o website educasaocarlos e o sistema de busca sibi. A ausência de uma API unificada e documentada para consulta do acervo exigiu da equipe um esforço adicional de mapeamento e adaptação dos dados disponíveis, de modo a viabilizar a funcionalidade de Consulta de Disponibilidade sem comprometer a integridade das informações já existentes.

No âmbito técnico, a escolha pelo React Native em conjunto com o Expo trouxe ganhos relevantes de produtividade, mas também impôs limitações específicas, sobretudo na integração de recursos nativos de geolocalização, que demandaram ajustes nas permissões e no comportamento entre as plataformas iOS e Android testadas via Expo Go.

A implementação das camadas de segurança também representou um desafio significativo. A configuração do gerenciamento de sessão sem persistência local (persistSession: false) exigiu o desenvolvimento de um AuthContext customizado capaz de lidar corretamente com o ciclo de vida do token em memória, garantindo que o usuário não fosse deslogado de forma inesperada durante a navegação, ao mesmo tempo em que se reduzia a superfície de ataque a vetores de XSS. De forma semelhante, a definição das políticas de Row Level Security (RLS) no Supabase demandou diversos testes para assegurar que o isolamento de dados entre usuários não comprometesse o funcionamento de funcionalidades compartilhadas, como o calendário de eventos e a curadoria de notícias.

Por fim, a coordenação entre os membros da equipe, com diferentes níveis de familiaridade com o ecossistema React/Supabase, exigiu alinhamento constante de padrões de código e de fluxo de trabalho no repositório, de modo a manter a consistência da base de código compartilhada entre web e mobile.

7.2 Aprendizados

O desenvolvimento do projeto proporcionou aprendizados relevantes tanto em aspectos técnicos quanto em aspectos de processo.

Do ponto de vista técnico, a equipe consolidou conhecimentos práticos sobre autenticação e autorização baseadas em JWT, compreendendo na prática as diferenças entre validação no cliente e validação no servidor, bem como a importância de mecanismos como Row Level Security para garantir a segurança de dados em aplicações que utilizam Backend as a Service (BaaS). A experiência também reforçou a relevância de cabeçalhos de segurança HTTP (como CSP, X-Frame-Options e X-Content-Type-Options) como camada complementar de proteção, frequentemente subestimada em projetos acadêmicos.

A adoção do React Native evidenciou, na prática, as vantagens do reaproveitamento de conhecimento entre plataformas web e mobile, mas também deixou claro que decisões de arquitetura tomadas na versão web — como o modelo de gerenciamento de estado e a estrutura de chamadas às APIs — impactam diretamente a viabilidade e o esforço de portabilidade para o ambiente mobile.

No campo de processo, o projeto reforçou a importância de priorizar funcionalidades a partir das personas, evitando o desenvolvimento de recursos de baixo impacto para o público-alvo principal (cidadãos, estudantes e participantes de eventos) em detrimento de funcionalidades essenciais. A definição tardia da Tela de Perfil, por exemplo, ilustrou como o refinamento contínuo de requisitos, mesmo após a prototipação inicial em Figma, é parte natural e necessária do processo de desenvolvimento ágil.

Por fim, a elaboração de casos de teste estruturados para as validações de cliente (Seção 5.4.3) evidenciou o valor de formalizar critérios de aceite antes da implementação, contribuindo para reduzir ambiguidades na definição de comportamento esperado e facilitando a verificação posterior da qualidade do software entregue.

A Link da Apresentação em PowerPoint

Clique aqui para acessar a apresentação